COMP 512 - Programming Assignment 2

Karl Wehbe, Ion Basha

May 22, 2026

1 Introduction

In this project, we implemented a Paxos consensus protocol to achieve total-order broadcast in a distributed system. The goal was to build a fault-tolerant system that guarantees all processes agree on the same sequence of moves, even in the presence of failures. This report summarizes the architecture and failure handling.

2 Architecture and Design

- **Proposer:** Runs Prepare, Accept, and Confirm with unique ballots. Adopts the highest accepted value in Prepare to ensure safety. After majority acceptance, sends CONFIRM, unblocks the waiting thread, and moves to the next round.
- **Acceptor:** Tracks round state and responds based on ballot ordering. After CONFIRM, buffers values and releases them in order, enforcing total order despite message reordering.
- **Message Queues:** Three blocking queues coordinate communication:
  - `outgoing`: app → Proposer which carries client requests (proposed values) from the application thread to the Proposer for consensus.
  - `incoming`: Acceptor → app which delivers confirmed, totally ordered values back to the application once they have been chosen.
  - `proposerQueue`: Acceptor → Proposer which transports PROMISE, REFUSE, and CONFIRM messages to the Proposer during Paxos rounds.
- **Paxos Message:** Communication uses `PaxosMessage` with seven types: PROPOSE, PROMISE, REFUSE, ACCEPT, ACCEPTACK, DENY, CONFIRM. Messages include round, ballot, previously accepted ballot, value, sender and destination IDs.

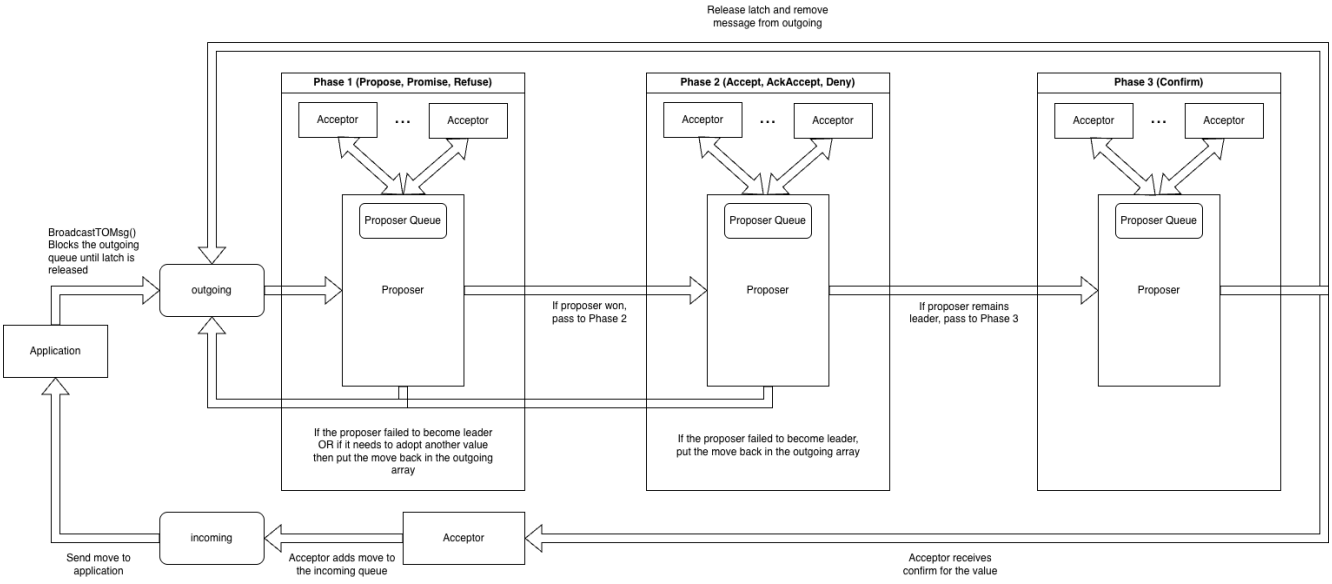


Figure 1: System diagram for Paxos (High level)

Ensuring Total Order Across All Processes

When multiple processes propose simultaneously, Paxos guarantees exactly one wins round n. Simultaneous moves are guaranteed to reach all game instances identically through three mechanisms:

1. **Round-based sequencing**: Each Paxos instance corresponds to one position in the total order. Processes start at round 0 and increment synchronously as values are decided.
2. **Ordered delivery**: The Acceptor's `TreeMap` buffer ensures values are delivered strictly in round order. If round 5's CONFIRM arrives before round 4's, it remains buffered until round 4 completes.
3. **Broadcast CONFIRM**: Once majority accepts, the winning Proposer broadcasts CONFIRM to *all* processes (not just the majority), ensuring even minority processes receive the decision. The losers receive the CONFIRM message, abort their current attempt, and retry in round $n + 1$.

Fairness Under High Load

Under high contention, no single process dominates move submission due to:

1. **Exponential backoff with jitter**: Failed proposers retry after a small delay which prevents synchronized collisions and gives other processes opportunity to win.
2. **Unique ballot IDs**: The unique ballot ID ensures no two processes generate identical ballots. Higher counters win, but the port acts as a tiebreaker among same-counter proposals, distributing leadership naturally.
3. **Application-level flow control**: `broadcastTOMsg(val)` blocks until majority acceptance. A process cannot flood proposals because it must wait for its previous move to be accepted before submitting the next. This rate-limits each process proportionally.
4. **Early abort**: Upon seeing a CONFIRM message, the Acceptor will forward that information to it's Proposer who will immediately abandon its attempt rather than completing unnecessary protocol rounds, reducing wasted bandwidth and allowing faster turnover.

Minority Process Convergence

Even though `broadcastTOMsg()` only waits for a majority, all processes eventually receive chosen values because:

1. **Confirm to all:** The winning proposer broadcasts CONFIRM to every process via `gcl.broadcastMsg()`, not just those that replied.
2. **Buffered ordering:** Each acceptor buffers out-of-order CONFIRM messages in a `TreeMap` and delivers them once earlier rounds arrive, so processes that miss rounds still apply values in correct order.
3. **Reliable delivery:** GCL ensures reliable delivery so acceptors are guaranteed to eventually receive messages.

Failure Handling

1. **AFTER SEND PROPOSE:** If a proposer crashes after sending PROPOSE, acceptors may have responded, but the proposer never collects replies. Other proposers eventually timeout and start a new round.
2. **AFTER BECOMING LEADER:** If a proposer crashes after receiving promises but before sending ACCEPT, no value has been chosen yet. Acceptors have only recorded a higher ballot, so another proposer can still initiate a new proposal and continue normally.

3. **After Value Accept:** If a proposer crashes after a majority accepts a value but before broadcasting CONFIRM, acceptors still retain the accepted value. When another proposer sends proposals, acceptors return this value. The new proposer sets aside its own pending move and adopts the previously accepted value and tries to become leader.

4. **RECEIVE PROPOSE:** If an acceptor crashes before replying to PROPOSE, it simply does not respond. The proposer continues as long as enough other acceptors reply and majority is alive.

5. **AFTER SEND VOTE:** If an acceptor crashes after sending a PROMISE, that promise still counts toward the decision. The acceptor no longer participates, but remaining processes complete the round as long as a majority is alive.

3 Collaboration

We collaborated in person across three meetings held on the third floor of the Trottier building. Between meetings, we used WhatsApp to coordinate progress and discuss design decisions. Ion focused mostly on the performance evaluation and contributed to parts of the Paxos implementation. Karl focused mostly on the Paxos algorithm. We both collaborated together to debug and resolve issues throughout the project.